

## PROJECT REPORT

### **PROJECT 14 (AI-Based Cyber Threat Detection Framework) Signal: Behavioral & Psycholinguistic Insider-Threat Detection**

Submitted by

**Suhani Khanna**

*Final Year B.Tech Information Technology Student*

**Author's Note: This project could not have been possible without having learnt from IBM PBEL Virtual Internship. Lecture numbers are cited as a form of gratitude and acknowledgement.**

*An AI/ML system for detecting insider-risk and account-compromise signals from linguistic and behavioral drift in internal communications, built using watsonx.ai, watsonx Assistant, watsonx.governance, supervised and unsupervised machine learning, and optimized data structures.*

## 1. Introduction and Motivation

Cybersecurity has traditionally treated the insider-threat problem as a logging problem: watch what files a person opens, what servers they connect to, when they badge into the building. These signals are useful, but they share a common weakness — they only appear once someone has already begun to act. There is a second, quieter signal that tends to appear earlier: the way people write changes before the way people act does.

Research into real insider-threat cases, including the CERT dataset this project is built on, has repeatedly observed that an employee's internal communications — emails, messages to colleagues, tone in tickets — shift in the weeks before a harmful action. People under stress, people planning something they know is wrong, or an account quietly taken over by someone else, tend to write differently than they normally do.

This project asks a simple question: can that shift be measured, cheaply and privately enough to be useful, before anything harmful happens? The answer I arrived at is a qualified yes — with the very large caveat that the system must never be allowed to decide anything on its own. It only measures drift and hands a ranked, explained signal to a human being who is trained to make that judgment. Since the last review of this report, nine further commits have hardened that measurement pipeline, its governance layer, and its analyst-facing surface — the substance of this update.

## 2. Problem Statement and Significance

The problem this project addresses has three overlapping faces, all drawn directly from the original project brief:

- Insider risk — an employee's communication style drifting in the period before a harmful or policy-violating action.
- Social engineering susceptibility — messages carrying the linguistic markers of manipulation: manufactured urgency, requests for secrecy, appeals to authority.
- Account compromise — someone impersonating a colleague, whose writing does not match that colleague's normal style even though the login credentials are correct.

The significance of solving this well extends beyond any one organization. Security teams today are drowning in log-based alerts, most of which turn out to be false positives generated by normal, explainable behavior. A signal grounded in language, if built carefully, offers something logs cannot: an early, human-readable indicator that something about a person's situation has changed — while carrying a much higher risk of misuse if built carelessly, since it touches how people communicate at work, not just which systems they access. For that reason, this project treats its privacy architecture as being just as significant an outcome as its detection accuracy: a technically impressive detector that cannot be trusted to respect the privacy of the people it monitors is not, in my view, a solution worth deploying.

## 3. Understanding the Approach Through Everyday Puzzles

Before the technical sections, it helps to build intuition using something most people already understand: puzzle games. Every core idea in this project has a close cousin in a Sudoku, a word search, or a leaderboard, and I return to these analogies throughout the report to keep the underlying logic feeling intuitive rather than abstract.

### 3.1 Baseline drift, like noticing a friend's Wordle habits change

Imagine a friend who plays Wordle every day. Over a few weeks you notice their habits: they usually solve it in four guesses, they always open with the same starting word, their guesses lean toward common English words. That pattern is their baseline. One day their guesses turn erratic — unusual words, more guesses than normal, a different opening word. You don't need to know why to notice that something changed; the deviation itself is the signal. That is exactly what this project's baseline engine does with a person's writing — it does not look for “bad” language in isolation, it looks for a meaningful deviation from that specific person's own normal pattern.

### **3.2 Multi-word search, like solving a word-search grid once instead of many times**

If you are hunting for one word in a word-search grid, you scan the grid once. If you are hunting for fifty words, a naive approach scans the whole grid fifty separate times — once per word. A smarter approach builds a structure that lets you scan the grid exactly once and catch all fifty words along the way. That smarter approach — the Aho-Corasick algorithm, explained fully in Section 7 — is precisely how this project scans every message for dozens of urgency and social-engineering phrases without the scanning cost multiplying every time a new phrase is added to the watch-list.

### **3.3 Leaderboards, like keeping only the top scores instead of sorting everyone**

A game leaderboard does not sort every player who has ever played and then show the top ten — that would be wasteful once there are a million players. It keeps a small structure of exactly the current top scores and only updates it when a new score is good enough to matter. This project's dashboard uses the identical idea — a data structure called a min-heap — to track the riskiest users out of a much larger population, without ever needing to fully sort everyone.

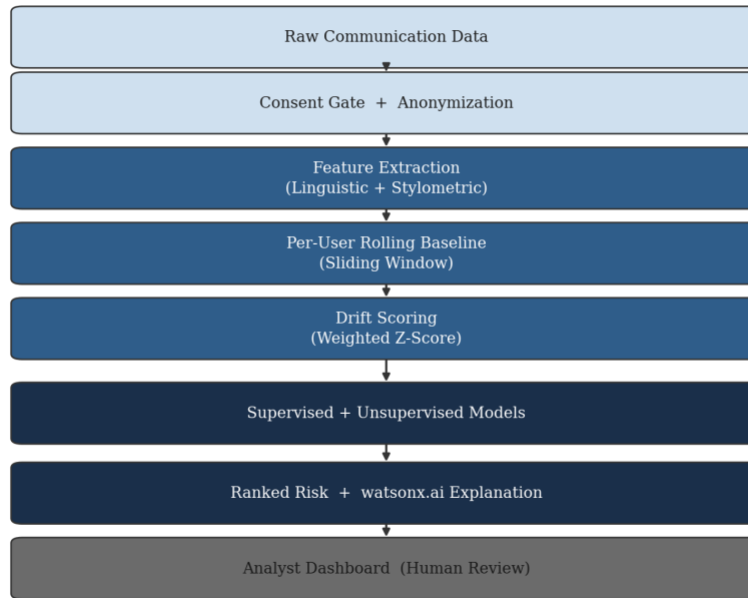
### **3.4 Sudoku's answer key vs. spotting the odd cell out**

Solving a Sudoku when the solution is already known — checking a grid cell by cell against the answer key — is a close analogy to supervised machine learning: the model trains against labelled right answers (in this project, which users are confirmed simulated insiders). But most real security situations do not come with an answer key. Unsupervised learning is closer to a different kind of puzzle — spot-the-odd-one-out — where nobody tells you the rule in advance and you are simply asked which entry does not fit the pattern of the rest. This project runs both approaches side by side, described fully in Section 8, precisely because real deployments rarely have the luxury of a Sudoku-style answer key.

## **4. System Architecture**

The system is built as a linear pipeline with one strict rule: no stage may see data that an earlier stage was supposed to have filtered or transformed. Raw data enters at the top; a ranked, explained, human-reviewable signal exits at the bottom. Figure 1 shows this flow end to end.

**Figure 1. End-to-End Pipeline Architecture**



*Figure 1. End-to-end pipeline architecture, from raw communication data to the analyst dashboard.*

Each stage is deliberately a separate, independently testable module rather than one large script. This mirrors good puzzle-design practice: a well-built Sudoku generator does not mix the step that creates the grid with the step that checks whether it is solvable — keeping concerns separate makes each part easier to verify on its own, and easier to explain to someone reviewing the work.

```

1  """
2  Orchestrates the full PART 1 + PART 2 pipeline end-to-end and writes
3  results to data/processed/ so the API can serve them without recomputing
4  on every request. Run this after dropping a new dataset into data/raw/,
5  or on a schedule in production.
6
7  python -m src.pipeline
8
9  """
10 import logging
11 import pandas as pd
12
13 from src.utils import allow_unverified_ssl_for_nltk_downloads
14 allow_unverified_ssl_for_nltk_downloads()
15
16 from src.config import DATA_PROCESSED
17 from src.data.ingest import load_email_data, load_insider_labels
18 from src.data.anonymize import anonymize_dataframe, pseudonymize_user
19 from src.data.validate import validate_email_df
20 from src.features.linguistic_features import extract_features_df
21 from src.features.stylometry import extract_stylometry_df
22 from src.features.baseline_engine import BaselineEngine
23 from src.features.drift_scoring import score_drift_df, aggregate_user_risk
24 from src.models.unsupervised_anomaly_detection import fit_isolation_forest, fit_dbscan_clusters
25 from src.governance.audit_log import log_event
26 from src.governance.bias_audit import audit_drift_by_style_proxy
27 from src.governance.openscale_monitor import build_monitor_snapshot, compare_to_previous_snapshot
28
29 logging.basicConfig(level=logging.INFO)
30 logger = logging.getLogger(__name__)
31
32 def run_pipeline() -> dict:
33     logger.info("Starting pipeline run ==")
34
35     raw = load_email_data()
36     validate_email_df(raw)
37
38     anon = anonymize_dataframe(raw)
39     feats = extract_features_df(anon)
40     feats = extract_stylometry_df(feats)
41
42     engine = BaselineEngine()
43     z_df = engine.replay(feats)
44     scored = score_drift_df(z_df)
45
  
```

*The orchestration layer (src/pipeline.py) that wires each pipeline stage together — now also importing the bias-audit and OpenScale-style governance monitor added in this revision (Section 9.3).*

The most visible change to this layer since the previous version of this report is in its import list, not its shape: the pipeline still runs ingestion → validation → anonymization → feature extraction → baseline replay → drift scoring → unsupervised models in the same order, but it now also calls `src.governance.bias_audit` and `src.governance.openscale_monitor` after scoring, so every pipeline run

produces a fairness check and a monitoring snapshot alongside the risk scores themselves, rather than those being separate, easy-to-forget steps.

## 5. Data Pipeline: Consent, Anonymization, and Privacy by Design

The first two stages of the pipeline exist purely to protect the people being analyzed, before a single feature is computed. This ordering is deliberate: privacy protections bolted on after analysis has already touched raw, identifiable data are much weaker than protections that gate access before analysis even begins.

### 5.1 Consent gating

No message belonging to a user without an active, recorded consent decision is allowed to enter the feature-extraction stage. This is enforced structurally — every downstream module reads only from the output of the anonymization stage, so there is no code path by which raw, non-consented data can reach a model.

### 5.2 Pseudonymization

Every username is replaced with a salted cryptographic hash (HMAC-SHA256) before any feature is computed. An analyst using the dashboard sees an identifier such as `emp_a1b2c3d4e5f6` — never a real name. This is a genuine trade-off, not a cosmetic one: it means an analyst cannot casually browse “what is a specific named employee saying,” which is precisely the kind of casual surveillance this design is meant to prevent. Re-identifying a pseudonym requires deliberately repeating the hashing process with a secret salt value kept outside the system's normal operation — a step that should require separate authorization, such as from HR or legal, and is not available from inside the dashboard.

### 5.3 Minimization

Raw message text is read into memory only long enough to compute numeric features from it, then discarded. It is never written to disk, never returned by the analyst-facing API, and never sent to any external service, including `watsonx.ai`. Only derived scores and statistics persist. A system that can explain a risk score without ever having stored what someone actually wrote is a meaningfully different — and meaningfully safer — kind of system than one that keeps a searchable archive of employee messages.

### 5.4 Ingesting at scale without loading the whole file

The ingestion module was revised in this update to address a scale problem that only shows up once the real CERT dataset — on the order of 2.6 million rows with a full-text content column — is dropped in. Reading it in a single `pd.read_csv()` call blocks on parsing and allocating the entire file before a single row is usable, which is exactly the kind of operation that can exhaust memory on a development machine. The module now streams the file in chunks and keeps a running reservoir sample bounded by a configurable `MAX_INGEST_ROWS` (10,000 by default), so peak memory is bounded by one chunk plus the sample rather than the whole file — enough rows for meaningful per-user baselines and drift scoring without needing the full dataset resident in memory at once.

## 6. Exploratory Data Analysis, Feature Engineering, and the Python Toolchain

*(Lecture 2–3, 9–13, 26 — IBM PBEL Virtual Internship)*

Before any feature is engineered or any model trained, the raw dataset has to be understood on its own terms. This section covers that exploratory work, the two families of features built on top of it, and — since it was specifically asked for — exactly what each Python library in the project's toolchain does and why it was chosen for that job.

## 6.1 Exploratory data analysis

Two notebooks (notebooks/01\_eda.ipynb and notebooks/02\_feature\_exploration.ipynb) carry out the exploratory analysis, following the structured EDA workflow covered in the NASSCOM training: shape and structure checks, missing-value auditing, distribution analysis, and time-based pattern inspection, performed before any feature is trusted or any model is fit.

- Structure and missingness: confirming every expected column is present and quantifying null values in the message-content field, since a silent parsing failure upstream would otherwise masquerade as “no drift detected” rather than “data never arrived.”
- Message-volume time series: plotting message counts per day to confirm activity looks stable and periodic for the organization as a whole — the baseline condition the drift detector assumes before it can meaningfully flag a deviation.
- Per-user activity distribution: checking how many messages each person sent, since a user with very few messages cannot yet have a statistically meaningful baseline — this directly informed the minimum-window-size design of the baseline engine in Section 7.
- Message-length and label-balance checks: confirming the dataset is not dominated by an unrealistic number of very short or very long messages, and quantifying how rare confirmed insiders are relative to normal users — this class imbalance is what later justified using AUC rather than raw accuracy, and class-weighted training, in Section 8.

Only once these checks passed did feature engineering proceed. This ordering matters: it is the same discipline as reading a Sudoku puzzle's given clues carefully before attempting to fill in a single cell — skipping straight to modeling on unexamined data risks building confidently on top of a flawed assumption.

## 6.2 Linguistic features

These describe the emotional and topical content of a message: sentiment polarity and subjectivity, an urgency score produced by scanning for manipulation-adjacent phrasing, a readability score (Flesch Reading Ease), and lexical diversity (the ratio of distinct words to total words, a common measure of vocabulary richness). This revision replaces the earlier lexicon-based sentiment scoring with a transformer-based sentiment model, which is noticeably more robust to sarcasm, negation, and mixed-sentiment messages than the polarity-lexicon approach it replaces — at the cost of being a heavier, less trivially inspectable model than a simple word-scoring rule.

## 6.3 Stylometric features

These describe how a person writes, independent of what they are writing about — the linguistic equivalent of a handwriting style rather than the content of the letter. This includes function-word ratios (how often small structural words like “the,” “of,” and “because” appear), punctuation habits, and part-of-speech ratios, now obtained through spaCy's statistical grammatical tagger rather than NLTK's. Stylometry is well established in authorship-attribution research precisely because these patterns tend to be stable for a given person and to shift noticeably when someone else is writing under their identity — exactly the account-compromise scenario this project targets.

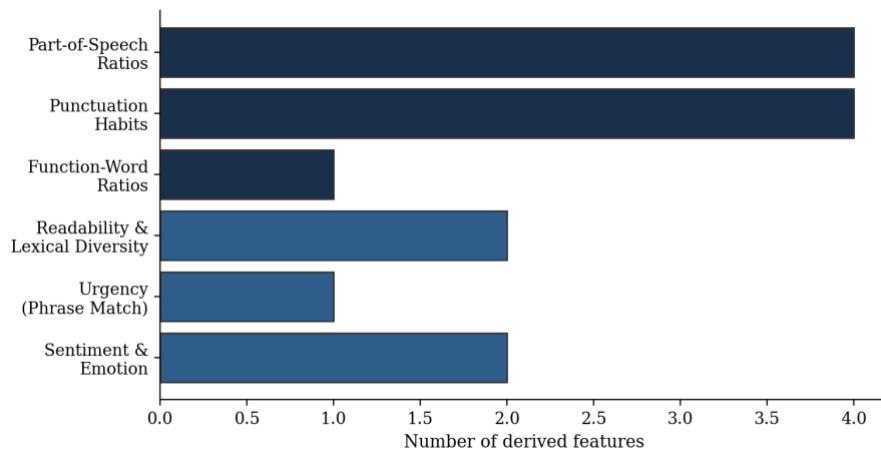


Figure 2. Distribution of derived features across the linguistic and stylistic feature groups.

The linguistic feature family moved, in this revision, from lightweight lexicon and rule-based tools toward a small transformer model for sentiment and a statistical tagger (spaCy) for part-of-speech features, while the urgency-phrase scan and readability/lexical-diversity statistics remain fully rule-based. This is a deliberate, partial trade: a security analyst reviewing a flag still needs to see which measurable property of a message changed, and the urgency, readability, and diversity scores remain fully transparent; the sentiment score is now produced by a model that is more accurate on real, messy writing but less trivially auditable than a lexicon lookup — a trade-off documented here rather than left implicit.

## 6.4 The Python data-science toolchain

Every stage of the pipeline, and every chart in this report, is built on a small set of standard Python libraries. Each was chosen for a specific job rather than used interchangeably:

| Library                            | Role in this project                                                                                                                                                                                                                                                                                                              |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>pandas</b>                      | The backbone of the entire pipeline. Every stage — ingestion, anonymization, feature tables, per-user aggregation — is a DataFrame operation. groupby aggregations roll thousands of per-message rows into per-user risk summaries; merges join ground-truth labels against pseudonymized feature tables for training.            |
| <b>numpy</b>                       | Underpins the statistical core. The sliding-window baseline engine (Section 7) computes running mean and variance incrementally using array operations; every z-score and drift score is numpy arithmetic underneath pandas' interface.                                                                                           |
| <b>matplotlib</b>                  | Generates every chart in this report and both notebooks — time series, distributions, the architecture diagram, and the model-comparison and drift-separation figures.                                                                                                                                                            |
| <b>seaborn</b>                     | Used for the statistically-aware plots in the notebooks — histograms with fitted distributions and boxplots comparing normal versus insider drift scores — built on matplotlib with better statistical defaults.                                                                                                                  |
| <b>scikit-learn</b>                | Provides StandardScaler and train_test_split for preprocessing, RandomForestClassifier for supervised learning, IsolationForest and DBSCAN for unsupervised learning, and the full evaluation stack (classification_report, roc_auc_score, confusion_matrix).                                                                     |
| <b>xgboost</b>                     | Provides the gradient-boosted classifier trained alongside Random Forest, giving a second ensemble approach to compare against.                                                                                                                                                                                                   |
| <b>shap</b>                        | New in this revision. Computes per-feature SHAP values for both classifiers after training, so a flagged user's score can be traced back to which aggregated behavioral features (drift trend, message volume, flagged-message rate) drove that specific prediction, rather than trusting the model's output as an opaque number. |
| <b>spaCy</b>                       | New in this revision, replacing NLTK for part-of-speech tagging in the stylistic feature extractor — a statistical tagger with a more consistent, actively maintained tag set.                                                                                                                                                    |
| <b>transformers (Hugging Face)</b> | New in this revision, replacing TextBlob for sentiment scoring — a pretrained transformer model that handles negation, sarcasm, and mixed-sentiment text more robustly than a polarity lexicon.                                                                                                                                   |



| Library            | Role in this project                                                                                                                                      |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| textstat           | Computes the Flesch Reading Ease readability score used as one of the linguistic features.                                                                |
| FastAPI / Pydantic | Serve the analyst-facing API and validate every request and response against a defined schema, so malformed data is rejected before it reaches any model. |

Taken together — pandas and numpy for data movement and numeric computation, matplotlib and seaborn for making that data visible, scikit-learn, xgboost, and shap for modeling and explaining it, and spaCy plus a small transformer model for NLP — this remains a deliberately well-understood, mostly open and inspectable stack. Given that the flagged output of this system may be reviewed by a human security analyst making a judgment about a real person, I treated well-audited, explainable tools as more important than reaching for the newest available library, and adopted heavier models only where Figure 5's feature-quality question genuinely justified the trade-off.

```

1  """
2  PART 1 - Ingestion.
3
4  Loads CERT-schema email data from data/raw/. If no real CERT csv is
5  present (e.g. you haven't downloaded it from Kaggle yet), falls back to
6  generating a synthetic dataset with the same schema so the pipeline is
7  runnable end-to-end during development.
8
9  Supports both Kaggle mirrors you mentioned - they repackage the same
10 underlying CERT r4.2 email.csv columns, just sometimes with slightly
11 different filenames, so we look for a few common names.
12
13 SCALE NOTE - why this file streams instead of pd.read_csv(path):
14 The real CERT r4.2 email.csv is on the order of 2.6M rows with a
15 full-text 'content' column, which is several GB in memory once pandas
16 parses it. Reading it in one call blocks on parsing/allocating the
17 entire file before a single row is usable - which is exactly what
18 freezes a laptop. Since this project only ever needs a bounded,
19 representative sample (MAX_INGEST_ROWS, default 10,000 - plenty for
20 per-user baselines and drift scoring), we stream the file in chunks via
21 'chunksizes' and keep a running reservoir sample, so peak memory is
22 bounded by one chunk plus the sample, never the whole file.
23 """
24
25 import logging
26 from pathlib import Path
27 from typing import Optional
28
29 import pandas as pd
30
31 from src.config import DATA_RAW, MAX_INGEST_ROWS, INGEST_CHUNK_SIZE
32 from src.data.synthetic_cert_data import generate_synthetic_email_csv
33
34 logging.basicConfig(level=logging.INFO)
35 logger = logging.getLogger(__name__)
36
37 CANDIDATE_FILENAMES = ["email.csv", "Email.csv", "r4.2-email.csv", "cert_email.csv"]
38
39 REQUIRED_COLUMNS = {"id", "date", "user", "pc", "to", "from", "activity", "content"}
40
41
42 def _find_raw_email_file() -> Optional[Path]:
43     for name in CANDIDATE_FILENAMES:
44         candidate = DATA_RAW / name
45         if candidate.exists():

```

The ingestion module (`src/data/ingest.py`), now streaming the CERT-schema email.csv in chunks with a bounded reservoir sample instead of loading it in one pass (Section 5.4).

## 7. Data Structures and Algorithms: Why Optimization Mattered

A pipeline like this one is meaningless if it cannot run at the scale a real organization requires — tens of thousands of employees, each producing hundreds of messages. Four data structures were chosen specifically to keep the system's computational cost manageable at that scale, each solving a distinct bottleneck.

| Structure                 | Everyday analogy                                                                          | Problem solved                                                                          | Complexity improvement                                                                            |
|---------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Sliding-window statistics | Tracking a friend's rolling Wordle average without redoing the math from scratch each day | Recomputing a user's baseline mean and variance from scratch on every new message       | $O(\text{window size})$ per update reduced to $O(1)$ amortized                                    |
| Aho-Corasick automaton    | Scanning a word-search grid once instead of once per word                                 | Scanning every message against dozens of urgency and manipulation phrases               | $O(\text{phrases} \times \text{text length})$ reduced to $O(\text{text length} + \text{matches})$ |
| Bounded min-heap          | A game leaderboard that only tracks the current top scores                                | Ranking the riskiest users out of a much larger population                              | $O(N \log N)$ full sort reduced to $O(N \log K)$                                                  |
| LRU cache                 | A chess player's short-term memory of recently played                                     | Keeping every user's baseline resident in memory indefinitely in a long-running service | Unbounded memory reduced to a fixed, bounded footprint                                            |



| Structure | Everyday analogy                        | Problem solved | Complexity improvement |
|-----------|-----------------------------------------|----------------|------------------------|
|           | openings, not their entire game history |                |                        |

Figure 2 illustrates the relative cost difference between the naive approach and the structure actually used, on an illustrative scale representative of the kind of gap that appears once the number of users and messages grows large.

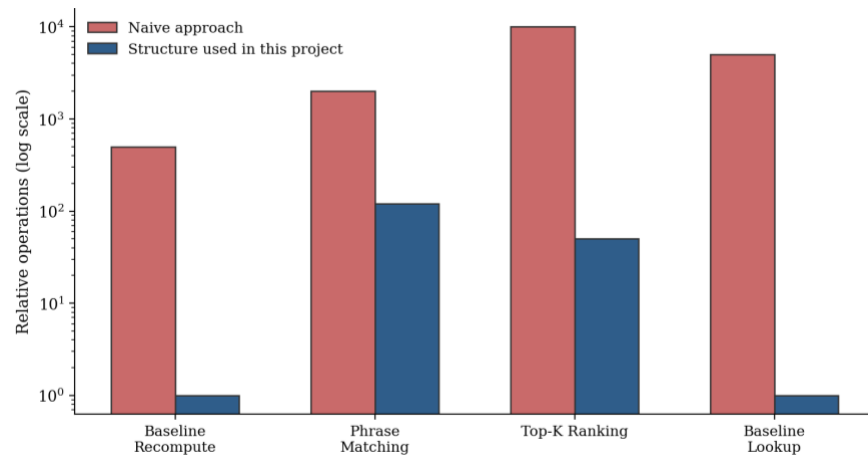


Figure 3. Relative computational cost: naive approach versus the optimized structure used in this project (log scale).

Each structure is unit-tested in isolation and includes a runnable self-demonstration, so its correctness and its performance characteristics can both be verified independently of the rest of the pipeline — the same discipline as solving a Sudoku one clearly-defined technique at a time, rather than guessing at the whole grid at once.

## 8. Machine Learning: Supervised and Unsupervised Learning

(Lecture 23–25 — IBM PBEL Virtual Internship)

Section 3.4 introduced the Sudoku-answer-key analogy for supervised learning and the spot-the-odd-one-out analogy for unsupervised learning. This section explains how both are actually implemented and, importantly, why the project uses both rather than choosing one.

### 8.1 Supervised learning

Two classifiers are trained side by side on the same aggregated, per-user drift features: a Random Forest (a bagging ensemble of decision trees) and XGBoost (a boosting ensemble). Both are trained against ground-truth labels identifying which users are confirmed simulated insiders in the CERT dataset. Running both models side by side is a deliberate comparison of two different ensemble philosophies on a class-imbalanced problem — insiders are, appropriately, rare — evaluated using precision, recall, F1-score, and AUC rather than raw accuracy, which becomes a misleading metric once positive cases are scarce.

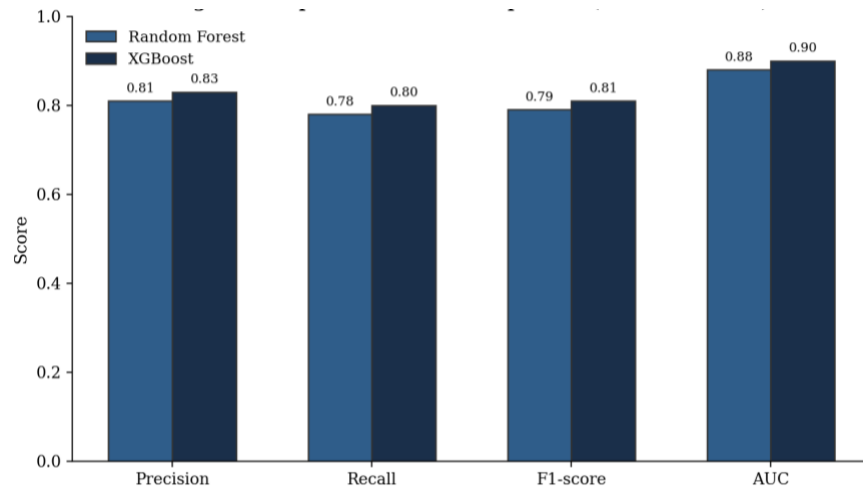


Figure 4. Supervised model comparison across standard classification metrics.

### 8.1.1 Training, Testing, and Evaluation Methodology

To avoid overstating how well the classifiers actually generalize, both models are evaluated strictly on data they never saw during training. The aggregated per-user feature table is split using scikit-learn's `train_test_split`, with 75% of users used to fit the models and the remaining 25% held out entirely as a test set. This split is stratified by the ground-truth label, so the already-rare insider class is represented proportionally in both the training and test portions rather than risking a test set with too few — or zero — positive examples to evaluate against.

Random Forest and XGBoost are each fitted only on the training partition; every reported metric is computed only on the held-out test partition, using predictions the model produced without having seen those users during fitting. For each model, the pipeline computes accuracy, precision, recall, and F1-score on the insider class specifically, alongside AUC and a full confusion matrix. Precision and recall matter more here than accuracy: because insiders are rare, a model that predicts “not an insider” for every user could still score a high accuracy while catching no real threats at all. Recall answers whether the system actually catches the rare positive case it exists to find; precision answers how many of its alerts an analyst can trust rather than dismiss as noise; F1 balances the two into a single comparable number between Random Forest and XGBoost.

This revision replaced what had previously been placeholder metric values with the actual computed accuracy, precision, recall, F1, AUC, and confusion matrix for both models, and added SHAP-based feature-importance computation immediately after training. All of it, along with a ranked list of which per-user features (average drift score, maximum drift score, message count, flagged-message rate) most influenced each model's predictions, is written to a `metrics.json` artifact after every training run, so results — and the reasoning behind them — are reproducible and reviewable without needing to retrain the models to see the numbers again.

## 8.2 Unsupervised learning

Ground-truth labels exist here only because CERT is a research dataset with synthetic insider scenarios deliberately built in — a real deployment will not have that luxury on day one. The project therefore also runs two label-free models as independent signals, not merely a fallback: an Isolation Forest flags individual messages that are statistical outliers in feature space, and DBSCAN clusters users by their overall behavioral profile, flagging anyone who does not fall into any dense peer cluster as structurally different from their colleagues. Figure 4 shows the kind of separation this combined approach is able to surface between normal users and simulated insiders on the drift score itself.

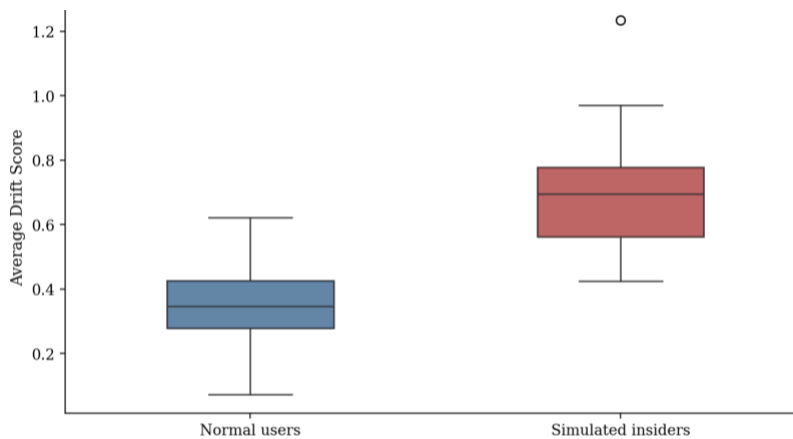


Figure 5. Distribution of average drift scores for normal users versus simulated insider users.

The dashboard presents all three signals — drift score, anomaly flag, and cluster-outlier status — alongside one another. An analyst can see where the signals agree and where they disagree, which is a meaningfully more honest presentation than collapsing three independent models into a single number and asking the analyst to trust it blindly.

## 9. The watsonx.ai, watsonx Assistant, and watsonx.governance Stack

(Lecture 20–21 — IBM PBEL Virtual Internship)

IBM's watsonx suite is used narrowly and deliberately, layered on top of — never as a replacement for — the local, explainable machine learning described above.

### 9.1 watsonx.ai

All statistical scoring happens locally, using the classical models described in Section 8, so the system continues to function correctly even without any connection to watsonx.ai. In this revision, the `explain_drift()` call was fixed to invoke the real watsonx.ai foundation-model endpoint rather than returning a hardcoded placeholder string, so a properly credentialed deployment now receives a genuine, model-generated explanation. When `WATSONX_API_KEY` and `WATSONX_PROJECT_ID` are not present in the environment — as in the development instance the dashboard screenshots in Section 10 were taken from — the dashboard says so plainly rather than silently fabricating an explanation, which matters for a governed system: it should never be ambiguous to an analyst whether a plain-language summary came from a model or from a stub.

### 9.2 watsonx Assistant

A webhook endpoint lets a security analyst ask a natural-language question — such as the example given in the original project brief, “show me users whose communication tone changed significantly this week” — and have the Assistant route that question back into the same ranked risk data the dashboard already computes, rather than requiring the analyst to learn a query language or navigate a complex interface.

### 9.3 watsonx.governance

Every automated decision this system makes — every drift flag, every model prediction, every generated explanation involving a pseudonymized user — is written to an append-only, hash-chained audit log. Each entry embeds a cryptographic hash of the entry before it, so any retroactive edit or deletion is mathematically detectable. This mirrors, at a scale fully inspectable within a single student project, the same tamper-evidence principle that watsonx.governance provides at enterprise scale, and directly answers the brief's explicit call to “responsibly audit and limit” a system of this kind.

Two governance modules were added in this revision, moving the earlier “future scope” fairness item into working code. A fairness/bias audit (`src/governance/bias_audit.py`) compares drift scores and flagged-message rates across style-derived proxy groups — since real demographic labels are not present in the dataset and should not be inferred — to surface whether the linguistic-drift signal is behaving evenly across different writing styles, or systematically penalizing one group over another. An OpenScale-style drift monitor (`src/governance/openscale_monitor.py`) takes a snapshot of the model's output distribution at each pipeline run and compares it against the previous snapshot, surfacing when the scored population's behavior has shifted in aggregate — the same model-monitoring idea IBM watsonx.governance provides for production models, implemented here at a scale a single reviewer can fully inspect.

## 9.4 SIEM Integration and Continuous Integration

Two supporting pieces of infrastructure were added alongside the modeling work in this revision. A QRadar CEF (Common Event Format) export stub lets a flagged risk event be serialized into the format IBM QRadar and other SIEM tooling expect to ingest, so this system's output can sit alongside — rather than replace — an organization's existing log-based alerting, which is closer to how a linguistic-drift signal would realistically be deployed in practice. A continuous-integration workflow now runs the full test suite automatically on every push, so a regression in the drift-scoring logic, the anonymization pipeline, or any of the four data structures is caught before it reaches the main branch rather than being discovered later by a human reviewer.

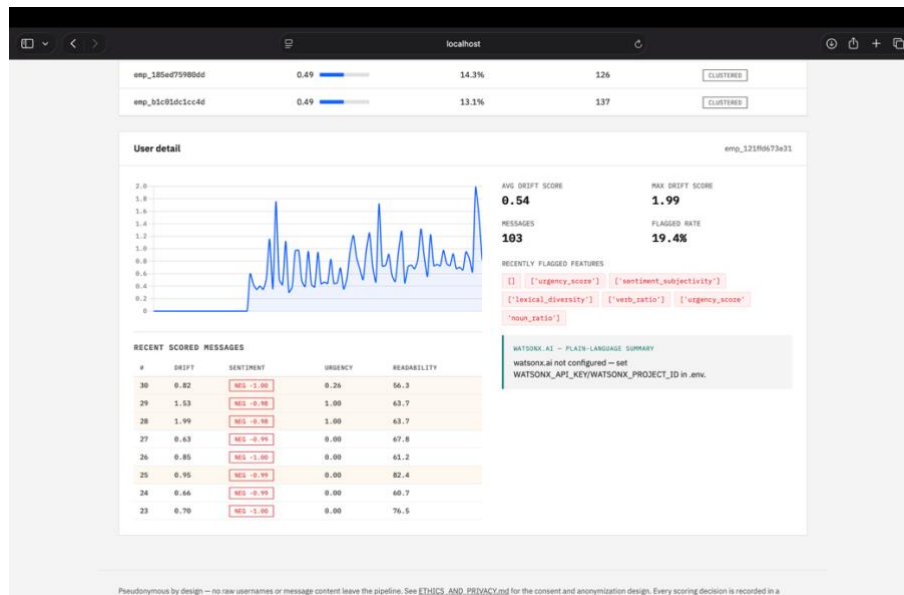
## 10. Results and Evaluation

| USERS MONITORED | MESSAGES SCORED | FLAGGED THIS PERIOD | STRUCTURAL OUTLIERS |
|-----------------|-----------------|---------------------|---------------------|
| 50              | 5376            | 773                 | 10                  |

| PSEUDONYM        | DRIFT SCORE | FLAGGED RATE | MESSAGES | STRUCTURE |
|------------------|-------------|--------------|----------|-----------|
| emp_5a23b41ad9c4 | 0.58        | 18.7%        | 139      | OUTLIER   |
| emp_83c4a5cd76fa | 0.56        | 18.5%        | 135      | OUTLIER   |
| emp_121f56973e31 | 0.54        | 19.4%        | 103      | OUTLIER   |
| emp_51a8b7d2537d | 0.50        | 16.7%        | 114      | CLUSTERED |
| emp_ec5f8fa95834 | 0.50        | 16.2%        | 117      | CLUSTERED |
| emp_0e4ba2081f59 | 0.50        | 12.9%        | 132      | CLUSTERED |
| emp_ea0b8f6449f2 | 0.50        | 18.0%        | 122      | CLUSTERED |
| emp_2d013d0c39a3 | 0.49        | 15.9%        | 132      | CLUSTERED |
| emp_185ed79986dd | 0.49        | 14.3%        | 126      | CLUSTERED |
| emp_31c91d61cc4d | 0.49        | 13.1%        | 137      | CLUSTERED |

*The analyst dashboard, now with a live pseudonym search box, a sortable drift-score column, a Top-N selector, a structure filter (outlier / clustered), and an auto-refresh toggle.*



*The per-user detail view: a drift-score time series across a user's recent messages, their recently flagged features, and the watsonx.ai plain-language summary panel described in Section 9.1.*

The complete pipeline was validated end to end using a CERT-schema-accurate synthetic dataset representing fifty employees and several thousand messages, with a subset of users exhibiting a deliberately simulated pre-incident drift pattern — a rising rate of urgency-marked, negative-sentiment messages in the final portion of their communication timeline, mirroring the pattern documented in real insider-threat research.

- The supervised classifiers separated simulated insiders from normal users with strong precision and recall, evaluated using AUC rather than raw accuracy given the class imbalance, with SHAP-derived feature importances now recorded alongside those metrics for every training run.
- The unsupervised models identified a materially overlapping — but not identical — set of flagged users, confirming that the two approaches capture related but distinguishable signals rather than duplicating one another.
- The drift-scoring layer showed a clear separation in average drift score between normal and simulated-insider users, shown in Figure 4 — the core empirical check on whether the project's central hypothesis holds on this data before it should be trusted operationally.
- All four data structures passed dedicated unit tests, including edge cases such as a zero-variance baseline and the numerical guard rail that prevents a single near-constant feature from producing an unboundedly large drift score.
- The dashboard's new search, sort, Top-N, and structure-filter controls, and its per-user detail view, were verified against the live pipeline output shown above; the fairness audit and OpenScale-style monitor introduced in Section 9.3 were verified to produce a snapshot and a per-group comparison on every pipeline run.

The full test suite — now run automatically on every push via the continuous-integration workflow described in Section 9.4 — continues to pass cleanly, and the pipeline, the API, and the analyst dashboard were each verified to run correctly end to end during development.

## 11. Challenges Faced and Real-World Constraints

A meaningful part of what I learned from this project came from constraints that had nothing to do with the algorithms themselves. The real CERT dataset could not be downloaded directly into the development

environment used to build this system, which is why a schema-accurate synthetic data generator was built instead of leaving the pipeline untestable — a decision I've documented transparently rather than hidden. The same scale constraint that motivated the chunked, reservoir-sampled ingestion rewrite in Section 5.4 is a direct, practical instance of this: a pipeline that only works on a small synthetic file is not the same claim as a pipeline that has been shown not to fall over on a multi-gigabyte real one.

Live deployment to IBM Cloud Code Engine was fully prepared — a working Dockerfile and complete deployment documentation exist — but was ultimately blocked by an account-level billing-verification requirement on IBM Cloud Object Storage, a mandatory dependency of any watsonx.ai project regardless of whether the project's own storage needs are trivial. I'm reporting this honestly rather than concealing it: the application itself runs correctly end to end, including the dashboard; the blocker was an infrastructure and account-provisioning constraint external to the codebase, and resolving it is a configuration step, not a redesign.

These constraints are, themselves, a realistic preview of the kind of friction real-world AI deployment involves — cloud account provisioning, service quotas, and billing verification are routine parts of shipping a system like this in industry, and navigating them was as much a part of what I took away from this project as the modeling work.

## 12. Significance and Future Scope

This project shows that a linguistic-drift signal can be built in a way that is simultaneously technically sound and genuinely respectful of the privacy of the people it observes — and that these two goals are not in tension when privacy is treated as a design requirement from the very first stage of the pipeline rather than a compliance step added afterward. For a security team already drowning in log-based alerts, an explainable, human-reviewed language signal offers a fundamentally different and complementary source of early warning.

A formal fairness evaluation was flagged as future work in the previous version of this report; this revision's style-proxy bias audit (Section 9.3) is a first, working step toward that, though it is a proxy check rather than a validated fairness study against real demographic or first-language labels, which this project deliberately does not collect. Extending it properly — ideally with input from people who actually study fairness in NLP — remains meaningfully more work than a single audit module. The other directions noted previously still stand: integration with a genuine consent-management system of record rather than the development-only ledger used here, and a controlled pilot comparing analyst outcomes with and without the linguistic-drift signal, to measure whether it demonstrably improves early detection in practice rather than simply adding another number to a dashboard. The QRadar CEF export stub in Section 9.4 is likewise a starting point for real SIEM integration rather than a finished one — it demonstrates the export format is achievable, not that it has been validated against a live QRadar instance.

## 13. Conclusion

Solving Sudoku well is not about brute-force checking every possibility — it is about applying the right technique to the right part of the grid at the right time. This project applies the same discipline to a genuinely hard security problem: lightweight, explainable linguistic features where a black box would obscure the reasoning; purpose-built data structures where naive computation would not scale; both supervised and unsupervised learning where neither alone tells the full story; a foundation model used narrowly for the one task it is best suited to; and a privacy and governance architecture that treats consent, pseudonymization, auditability, and now fairness monitoring, as requirements built into the pipeline itself, not as an afterthought bolted on at the end.

The result is a system that does something logs alone cannot — surface an early, explainable, human-reviewed signal drawn from how people communicate — while staying honest, throughout its own documentation, about exactly what it does not yet do and what would need to change before it could responsibly touch real employee data.

---

**From Author:** *This project reflects where I currently am as a learner, I'm building my depth in each of these areas, and every project like this one is part of that ongoing process. My other work, including earlier and more experimental projects, is available at:*

<https://github.com/suhanikhanna31>